

```
#importando as dependências e bibliotecas do projeto
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.compose import make_column_transformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import GridSearchCV
```

```
#declarando uma string com a url do dataset
#url = 'https://github.com/malegopc/DSBD/blob/main/Datasets/Churn/Churn_Modelling_2.csv' #dá erro pois temos que
url_dataset = 'https://raw.githubusercontent.com/malegopc/DSBD/main/Datasets/Churn/Churn_Modelling_2.csv'
```

```
#lendo o dataset
dataset = pd.read_csv(url_dataset, sep=",")
```

▼ Etapa A: Pré Processamento dos Dados

Essa etapa inicial corresponde à tarefa 5.1, na qual estive ausente. Aqui, iremos fazer os procedimentos de transformação, imputação e escalonamento para garantir um padrão ideal entre as variáveis do dataset

Adicionar aspas

```
#verificando o formato e estrutura geral do dataset
dataset.describe()
dataset.info()
dataset.head()
dataset.tail()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   RowNumber             10000 non-null  int64
1   CustomerId            10000 non-null  int64
2   Surname                10000 non-null  object
3   CreditScore            10000 non-null  int64
4   Geography              10000 non-null  object
5   Gender                 10000 non-null  object
6   Age                    10000 non-null  int64
7   Tenure                 10000 non-null  int64
8   Balance                10000 non-null  float64
9   NumOfProducts          9990 non-null   float64
10  HasCrCard              10000 non-null  int64
11  IsActiveMember         9986 non-null   float64
12  EstimatedSalary        10000 non-null  float64
13  Exited                  10000 non-null  int64
dtypes: float64(4), int64(7), object(3)
memory usage: 1.1+ MB
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard
9995	9996	15606229	Obijaku	771	France	Male	39	5	0.00	2.0	
9996	9997	15569892	Johnstone	516	France	Male	35	10	57369.61	1.0	
9997	9998	15584532	Liu	709	France	Female	36	7	0.00	1.0	
9998	9999	15682355	Sabbatini	772	Germany	Male	42	3	75075.31	2.0	
9999	10000	15628319	Walker	792	France	Female	28	4	130142.79	1.0	

```
#removendo as variáveis "RowNumber", "CustomerId", "Surname"
colunas_descartadas = ['RowNumber', 'CustomerId', 'Surname'] #lista de strings com dados a serem descartados
dataset = dataset.drop(columns=colunas_descartadas) #passando a lista como argumento
#porque é interessante e adequado remover essas colunas? não faz sentido que o modelo adote o ID de consumidor,
dataset.head()
```

	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary
0	619	France	Female	42	2	0.00	1.0	1	1.0	101348.88
1	608	Spain	Female	41	1	83807.86	1.0	0	1.0	112542.58
2	502	France	Female	42	8	159660.80	3.0	1	0.0	113931.57
3	699	France	Female	39	1	0.00	2.0	0	0.0	93826.63
4	850	Spain	Female	43	2	125510.82	1.0	1	1.0	79084.10

Próximas etapas:

[Gerar código com dataset](#)[New interactive sheet](#)

```
#verificando variáveis c dados ausentes
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   CreditScore            10000 non-null  int64   
1   Geography              10000 non-null  object   
2   Gender                 10000 non-null  object   
3   Age                    10000 non-null  int64   
4   Tenure                 10000 non-null  int64   
5   Balance                10000 non-null  float64  
6   NumOfProducts          9990 non-null   float64  
7   HasCrCard              10000 non-null  int64   
8   IsActiveMember         9986 non-null   float64  
9   EstimatedSalary        10000 non-null  float64  
10  Exited                  10000 non-null  int64   
dtypes: float64(4), int64(5), object(2)
memory usage: 859.5+ KB
```

```
#removendo as linhas com dados ausentes da variável "IsActiveMember"
dataset = dataset.dropna(subset=["IsActiveMember"])
```

```
#calculando a média da variável "NumOfProducts" para imputar
media_produtos = dataset['NumOfProducts'].mean()

print(f'Média Calculada: ', media_produtos)

#imputando
dataset['NumOfProducts'] = dataset['NumOfProducts'].fillna(media_produtos)
```

Média Calculada: 1.5301724137931034

```
#instanciando o codificador
label_encoder = LabelEncoder()

#ajustando o fit e o transform
dataset['Gender'] = label_encoder.fit_transform(dataset['Gender'])

#verificando o resultado na calda pois na cabeça só existem mulheres
dataset[['Gender']].tail()
```

	Gender	
9995	1	
9996	1	
9997	0	
9998	1	
9999	0	

```
#criando o transformador das colunas
col_transformer = make_column_transformer(
    (OneHotEncoder(drop='first', sparse_output=False), ['Geography']),
    remainder='passthrough'
)

#aplicando a transformação
dados_transformados = col_transformer.fit_transform(dataset)

#obtendo os novos nomes gerados pela scikit-learn
```

```
nomes_colunas = col_transformer.get_feature_names_out()
```

```
#reconstruindo o dataframe
```

```
dataset = pd.DataFrame(dados_transformados, columns=nomes_colunas)
```

```
#validando o resultado
```

```
dataset.head()
```

	onehotencoder__Geography_Germany	onehotencoder__Geography_Spain	remainder__CreditScore	remainder__Gender	re
0	0.0	0.0	619.0	0.0	
1	0.0	1.0	608.0	0.0	
2	0.0	0.0	502.0	0.0	
3	0.0	0.0	699.0	0.0	
4	0.0	1.0	850.0	0.0	

Próximas etapas:

[Gerar código com dataset](#)
[New interactive sheet](#)

```
#criando um array bidimensional que recebe o dataframe inteiro
```

```
X = dataset.iloc[:, :-1].values
```

```
#verificando o tipo e formato
```

```
print("Tipo da variável X:", type(X))
```

```
print("Formato do array X (Linhas, Colunas):", X.shape)
```

```
Tipo da variável X: <class 'numpy.ndarray'>
```

```
Formato do array X (Linhas, Colunas): (9986, 11)
```

```
#criando a variável com apenas a última coluna
```

```
y = dataset.iloc[:, -1].values
```

```
#verificando o resultado
```

```
print("Tipo da variável y:", type(y))
```

```
print("Formato do array y (Elementos,):", y.shape)
```

```
Tipo da variável y: <class 'numpy.ndarray'>
```

```
Formato do array y (Elementos,): (9986,)
```

```
#dividindo dados em conjunto de treino e teste para a próxima etapa
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
#exibindo o tamanho dos conjuntos e proporções
```

```
print("Dimensões dos Arrays")
```

```
print(f"X_train (Treino): {X_train.shape}")
```

```
print(f"X_test (Teste) : {X_test.shape}")
```

```
print(f"y_train (Alvos de Treino) : {y_train.shape}")
```

```
print(f"y_test (Alvos de Teste) : {y_test.shape}")
```

```
Dimensões dos Arrays
```

```
X_train (Treino): (7988, 11)
```

```
X_test (Teste) : (1998, 11)
```

```
y_train (Alvos de Treino) : (7988,)
```

```
y_test (Alvos de Teste) : (1998,)
```

```
#padronização e escalonamento com standardscaler
```

```
scaler = StandardScaler()
```

```
#ajustando fit da variável x de treino
```

```
X_train = scaler.fit_transform(X_train)
```

```
#transformando os dados de teste
```

```
X_test = scaler.transform(X_test)
```

```
#verificando o primeiro registro de treino p garantir a padronização
```

```
print("Primeiro registro de X_train padronizado:\n", X_train[0])
```

```
Primeiro registro de X_train padronizado:
```

```
[-0.57831397 -0.57368819  0.18512827  0.90852151  0.20142483  0.34024419
 -1.21200185  0.81334549 -1.55689361 -1.03335518  1.17467225]
```

▼ Etapa B: Aplicação de Modelos de Machine Learning

Na etapa B, o conteúdo utilizado corresponde exatamente à atividade 5.1, onde aplicaremos algoritmos de aprendizado de máquina

```
#criando um dicionário para armazenar os 4 tipos de algoritmos que serão implementados
#cada chave corresponde a um algoritmo e nos valores teremos conjuntos (sets) para armazenar os pesos, com os qu
algoritmos_otimizados = {
    "Regressão Logística": (
        LogisticRegression(class_weight='balanced', random_state=42, max_iter=2000),
        {'C': [0.1, 0.01, 0.001]} #testando diferentes forças de o
    ),
    "KNN": (
        KNeighborsClassifier(),
        {'n_neighbors': [3, 5, 7, 9, 11], 'weights': ['uniform', 'distance']} #testando n de vizinhos e pesos
    ),
    "SVM": (
        SVC(class_weight='balanced', random_state=42),
        {'C': [0.1, 1, 10], 'kernel': ['linear', 'rbf']} #tipos de margem matematica
    ),
    "Random Forest": (
        RandomForestClassifier(class_weight='balanced', random_state=42),
        {'n_estimators': [100, 200], 'max_depth': [10, 20, None]} #quantidade e profundidade das arvores
    )
}
```

```
#loop para percorrer o dicionário e testes
for nome, (modelo_base, parametros) in algoritmos_otimizados.items():

    #testador automatico -> gridsearch
    #validação cruzada e uso do
    grid = GridSearchCV(modelo_base, parametros, cv=5, scoring='accuracy', n_jobs=-1)

    #testa tds combinações e descobre a mlhr
    grid.fit(X_train, y_train)

    #modelo c melhor acurácia
    melhor_modelo = grid.best_estimator_

    #previsão e modelo vencedor
    y_pred = melhor_modelo.predict(X_test)

    #calculando metricas
    taxa_acerto = accuracy_score(y_test, y_pred)
    matriz_confusao = confusion_matrix(y_test, y_pred)

    #resultados
    print(f"Modelo: {nome}")
    print(f"Melhor configuração encontrada: {grid.best_params_}")
    print(f"Taxa de Acerto (Accuracy): {taxa_acerto * 100:.2f}%")
    print(f"Matriz de Confusão:\n{matriz_confusao}")
    print("-" * 50) #layout e separacao
```

```
Modelo: Regressão Logística
Melhor configuração encontrada: {'C': 0.01}
Taxa de Acerto (Accuracy): 69.92%
Matriz de Confusão:
[[1120  478]
 [ 123  277]]
-----
Modelo: KNN
Melhor configuração encontrada: {'n_neighbors': 11, 'weights': 'uniform'}
Taxa de Acerto (Accuracy): 83.83%
Matriz de Confusão:
[[1542   56]
 [ 267  133]]
-----
Modelo: SVM
Melhor configuração encontrada: {'C': 1, 'kernel': 'rbf'}
Taxa de Acerto (Accuracy): 80.18%
Matriz de Confusão:
[[1311  287]
 [ 109  291]]
-----
Modelo: Random Forest
Melhor configuração encontrada: {'max_depth': None, 'n_estimators': 200}
Taxa de Acerto (Accuracy): 85.94%
Matriz de Confusão:
[[1540   58]
 [ 223  177]]
-----
```

